-snoc function as shown below:

```
(-snoc list elements) ;; Syntax

(-snoc '(1 2 3) 4)
(1 2 3 4)
```

Given a separator character, you can create a new list with the separator inserted between the elements present in the input list using the *-interpose* function, as illustrated below:

```
(-interpose separator list) ;; Syntax

(-interpose ":" '("p" "q" "r"))
("p" ":" "q" ":" "r")
```

The *-interleave* function returns a new list with the first element from each list, followed by the second elements and so on. For example:

```
(-interleave lists) ;; Syntax

(-interleave '(1 2) '("a" "b"))
(1 "a" 2 "b")
```

The *-zip-with* function takes three arguments — a function, and two lists. It applies the function pairwise between each element in the input lists. An example is shown below:

```
(-zip-with function list1 list2) ;; Syntax

(-zip-with '+ '(1 2) '(3 4))
(4 6)
```

The *-zip* function can be used to return a list of lists, as demonstrated below:

```
(-zip lists) ;; Syntax


(-zip '(1 2) '(4 5))
((1 . 4) (2 . 5))
```

If you have lists with different lengths, then you can use a character to fill the shorter lists using the *-zip-fill* function, as shown below:

```
(-zip-fill fill-value lists) ;; Syntax

(-zip-fill 0 '(1 2 3) '(4 5))
((1 . 4) (2 . 5) (3 . 0))
```

The *-unzip* function takes a list of lists, and does the opposite of the *-zip* function. For example:

```
(-unzip lists) ;; Syntax

(-unzip '((1 2) (3 4) (5 6)))
((1 3 5) (2 4 6))
```

The *-cycle* function returns an infinite copy of a list by repeating the elements in the list. In the following example, the *-take* function is used to return only three elements:

```
(-cycle list) ;; Syntax

(-take 3 (-cycle '(1 2)))
(1 2 1)
```

The *-pad* function accepts a fill character value and appends it to the end of the shorter list. For example:

```
(-pad fill-value lists)  ;; Syntax

(-pad 0 '(1 2 3) '(4 5))
((1 2 3) (4 5 0))
```

You can compute the outer product of lists using the *-table* function. If you have two lists of dimensions *n* and *m*, then the outer product is the *n* x *m* matrix as shown below:

```
(-table function lists) ;; Syntax

(-table '* '(1 2 3) '(1 2 3))
((1 2 3) (2 4 6) (3 6 9))
```

If you want to specify a function for computing the outer product, then you can use the *-table-flat* function. An example is given below:

```
(-table-flat function lists) ;; Syntax

(-table-flat '* '(1 2 3) '(1 2 3))
(1 2 3 2 4 6 3 6 9)
```

The *-first* function takes a predicate function and a list, and returns the first element that matches the predicate. For example:

```
(-first predicate list) ;; Syntax

(defun even? (num) (= 0 (% num 2)))

(-first 'even? '(3 4 5))
4
```

The *-some* function also takes a predicate function and a list, and returns *True* if there is at least one element that matches the predicate. Otherwise, it returns *nil*. A couple of